

Milestone 3

Sprint 1 Report - NirogPath Healthcare Hub

This report includes the API tests written and executed for the **NirogPath Healthcare & OPD Prescription Hub** backend platform. It covers test setup design, endpoint inputs and output verifications, edge-case bug discoveries, changes implemented to resolve those issues, post-testing architecture improvements, structured user feedback gathered after demonstrating the platform, and the execution plan for Sprint 2.

Test Setup and Approach

Framework and Tools:

For the backend test suite, **pytest** is used alongside FastAPI's test utilities and `requests`. All tests execute against an isolated in-memory MongoDB environment powered by `mongomock-motor`, ensuring complete isolation from the production database. Sensitive external services (such as SMS/gateway triggers) are mocked, allowing fast, repeatable, and non-destructive local testing.

Database Isolation:

Each test module and fixture maintains strict database isolation. The shared fixture hierarchy in `conftest.py` resets and re-seeds state prior to running test cases. This guarantees that mutations in one test (e.g., wallet deductions, prescription creation) do not leak into or invalidate subsequent tests.

Fixture Hierarchy:

Automated seed fixtures (`patient_ctx`, `doctor_ctx`, `recep_ctx`, `prescription_setup`) perform automatic user authentication via `POST /api/auth/login` to obtain authentic JWT bearer tokens. This ensures that role checking, JWT decoding, and access control decorators are exercised on every protected route call.

API Test Cases

1. Successful patient registration returns token and ₹2000 initial wallet balance

POST /api/auth/register

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"name": "QA User", "email": "qa+patient101@nirog.in", "password": "testpassword123", "phone": "+919876543210"}
- **Header:** Authorization: (none – public endpoint)

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON Structure:** {"token": "<jwt_string>", "user": {"email": "qa+patient101@nirog.in", "role": "patient", "wallet_balance": 2000}}
- Password must never be returned in the response object

Actual Output:

- **HTTP Status Code:** 200 OK
- **JSON:** JWT token present, user role verified as "patient", wallet_balance initialized to 2000, password field excluded.

Result: Success

Snapshot of pytest function:

```
def test_register_new_patient():
    email = f"qa+{int(time.time()*1000)}@nirog.in"
    r = requests.post(f"{API}/auth/register", json={
        "email": email, "password": "test1234", "name": "QA User", "phone": "+911111111111"
    }, timeout=15)
    assert r.status_code == 200, r.text
    j = r.json()
    assert j["user"]["wallet_balance"] == 2000
    assert j["user"]["role"] == "patient"
    assert "token" in j
```

2. Writing E-Prescription with invalid frequency returns HTTP 400 validation error

POST /api/doctor/bookings/{id}/prescription

Inputs:

- **Request Method:** POST
- **Path Parameter:** booking_id of active consultation
- **JSON Payload:** {"medications": [{"name": "Amoxicillin", "dose": "500 mg", "frequency": "F00", "duration_days": 2}]}
- **Header:** Authorization: Bearer <doctor_token>

Expected Output:

- **HTTP Status Code:** 400 Bad Request
- **JSON Structure:** {"detail": "Invalid frequency F00. Must be one of: OD, BID, TID, QID"}

Actual Output (Before Fix):

- **HTTP Status Code:** 500 Internal Server Error — route crashed due to unhandled KeyError when computing medication schedule.

Result: Failed (Fixed Later)

Snapshot of pytest function:

```
def test_invalid_frequency_rejected():
    patient = _register_patient()
    doctor, slot = _pick_doctor_with_slot()
    d_ctx = _login_doctor_for(doctor)
    br = requests.post(f"{API}/bookings", json={"doctor_id": doctor["id"], "slot_time": slot},
                      headers=_h(patient), timeout=15).json()["booking"]
    requests.post(f"{API}/bookings/{br['id']}/arrive", headers=_h(patient), timeout=15)
    requests.post(f"{API}/doctor/bookings/{br['id']}/call-next", headers=_h(d_ctx), timeout=15)

    r = requests.post(f"{API}/doctor/bookings/{br['id']}/prescription",
                    json={"medications": [
                        {"name": "X", "dose": "1", "frequency": "F00", "duration_days": 2}
                    ]},
                    headers=_h(d_ctx), timeout=15)
    assert r.status_code == 400
```

3. Booking OPD consultation deducts token deposit from patient wallet

POST /api/bookings

Inputs:

- **Request Method:** POST
- **JSON Payload:** {"doctor_id": "doc-kavya-1", "slot_time": "10:30 AM"}
- **Header:** Authorization: Bearer <patient_token> (Initial balance: ₹2000)

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON Structure:** {"booking": {"status": "booked", "token_number": 1, "deposit_paid": 200}}
- Patient's remaining wallet balance must equal initial balance minus deposit (₹1800)

Actual Output:

- **HTTP Status Code:** 200 OK
- **JSON:** Booking created with status "booked", token assigned, wallet balance updated from ₹2000 to ₹1800.

Result: Success

Snapshot of pytest function:

```
def test_create_booking_and_wallet_deduction(fresh_patient):
    docs = requests.get(f"{API}/doctors", timeout=15).json()["doctors"]
    nisha = next(d for d in docs if "Nisha" in d["name"])
    slot = _find_free_slot(nisha["id"])
    initial_balance = fresh_patient["user"]["wallet_balance"]

    r = requests.post(f"{API}/bookings", json={"doctor_id": nisha["id"], "slot_time": slot},
                     headers=_h(fresh_patient), timeout=15)
    assert r.status_code == 200, r.text
    b = r.json()["booking"]
    assert b["status"] == "booked"
    assert b["token_number"] >= 1

    me = requests.get(f"{API}/auth/me", headers=_h(fresh_patient), timeout=15).json()["user"]
    assert me["wallet_balance"] == initial_balance - b["deposit_paid"]
```

4. Patient role cannot access doctor queue endpoints (RBAC Enforcement)

GET /api/doctor/queue

Inputs:

- **Request Method:** GET
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 403 Forbidden
- **JSON:** {"detail": "Role doctor required"}

Actual Output:

- **HTTP Status Code:** 403 Forbidden

Result: Success

Snapshot of pytest function:

```
def test_role_patient_cannot_call_doctor(patient_ctx):  
    r = requests.get(f"{API}/doctor/queue", headers=h(patient_ctx), timeout=15)  
    assert r.status_code == 403
```

5. Patient cannot view another patient's prescription (Data Privacy Access Control)

GET /api/prescriptions/{id}

Inputs:

- **Request Method:** GET
- **Path Parameter:** id of Patient A's prescription
- **Header:** Authorization: Bearer <Patient_B_token>

Expected Output:

- **HTTP Status Code:** 403 Forbidden
- **JSON:** {"detail": "Access denied to prescription"}

Actual Output (Before Fix):

- **HTTP Status Code:** 200 OK — route returned medical prescription data without checking if `prescription.patient_id == logged_in_user.id`.

Result: Failed (Fixed Later)

Snapshot of pytest function:

```
def test_prescription_access_control(prescription_setup):
    p = prescription_setup["prescription"]
    patient = prescription_setup["patient"]

    # Owner can view
    r = requests.get(f"{API}/prescriptions/{p['id']}", headers=_h(patient), timeout=15)
    assert r.status_code == 200

    # Unrelated patient must be denied
    other = _register_patient()
    r2 = requests.get(f"{API}/prescriptions/{p['id']}", headers=_h(other), timeout=15)
    assert r2.status_code == 403
```

6. Doctor calling next patient updates booking status to in_consultation

POST /api/doctor/bookings/{id}/call-next

Inputs:

- **Request Method:** POST
- **Path Parameter:** id of queued patient booking
- **Header:** Authorization: Bearer <doctor_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** {"status": "in_consultation"}

Actual Output:

- **HTTP Status Code:** 200 OK — booking status transitioned cleanly to "in_consultation".

Result: Success

Snapshot of pytest function:

```
def test_doctor_call_next_and_complete(doctor_ctx):
    q = requests.get(f"{API}/doctor/queue", headers=_h(doctor_ctx), timeout=15).json()["queue"]
    booked = next((b for b in q if b["status"] == "booked"), None)
    if booked:
        r = requests.post(f"{API}/doctor/bookings/{booked['id']}/call-next", headers=_h(doctor_ctx),
                        assert r.status_code == 200
```

7. Patient cancelling booking restores deposit amount back to wallet balance

POST /api/bookings/{id}/cancel

Inputs:

- **Request Method:** POST
- **Path Parameter:** id of active booking
- **Header:** Authorization: Bearer <patient_token>

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** {"message": "Booking cancelled"}
- Wallet balance after cancellation must equal balance before cancellation + deposit amount.

Actual Output:

- **HTTP Status Code:** 200 OK — booking cancelled, deposit credited back to patient wallet immediately.

Result: Success

Snapshot of pytest function:

```
def test_arrive_and_cancel_flow():
    email = f"qa+arr{int(time.time()*1000)}@nirog.in"
    reg = requests.post(f"{API}/auth/register", json={
        "email": email, "password": "test1234", "name": "Arrive QA"
    }, timeout=15).json()
    docs = requests.get(f"{API}/doctors", timeout=15).json()["doctors"]
    arjun = next(d for d in docs if "Arjun" in d["name"])
    slot = _find_free_slot(arjun["id"])
    r = requests.post(f"{API}/bookings", json={"doctor_id": arjun["id"], "slot_time": slot},
        headers=_h(reg), timeout=15).json()
    bid = r["booking"]["id"]
    dep = r["booking"]["deposit_paid"]

    ar = requests.post(f"{API}/bookings/{bid}/arrive", headers=_h(reg), timeout=15)
    assert ar.status_code == 200

    before = requests.get(f"{API}/auth/me", headers=_h(reg), timeout=15).json()["user"]["wallet_balance"]
    cn = requests.post(f"{API}/bookings/{bid}/cancel", headers=_h(reg), timeout=15)
    assert cn.status_code == 200
    after = requests.get(f"{API}/auth/me", headers=_h(reg), timeout=15).json()["user"]["wallet_balance"]
    assert after == before + dep
```


8. Hospital directory lists all partner hospitals with complete metadata & doctor counts

GET /api/hospitals

Inputs:

- **Request Method:** GET
- **Header:** Authorization: (none – public endpoint)

Expected Output:

- **HTTP Status Code:** 200 OK
- **JSON:** List of at least 3 hospitals, each containing required keys: {"id", "name", "city", "area", "rating", "reviews", "doctor_count", "specialties", "min_fee"}

Actual Output:

- **HTTP Status Code:** 200 OK — 3 hospitals returned with complete schema validation passing.

Result: Success

Snapshot of pytest function:

```
def test_list_hospitals():
    r = requests.get(f"{API}/hospitals", timeout=15)
    assert r.status_code == 200
    hospitals = r.json()["hospitals"]
    assert len(hospitals) >= 3
    required_fields = {"id", "name", "city", "area", "rating", "reviews", "image", "tags", "doctor_co
    for h in hospitals:
        missing = required_fields - set(h.keys())
        assert not missing, f"hospital {h.get('name')} missing fields: {missing}"
```

Bugs Found During Testing and Changes Made to Fix Them

1. Prescription Frequency Validation Error (HTTP 500 Route Crash)

Failing Test: `test_prescriptions.py::test_invalid_frequency_rejected`

Symptom: When a doctor entered a custom or non-standard dosage frequency string (e.g. "F00"), the server threw an unhandled `KeyError` exception inside the schedule calculation helper and returned an HTTP 500 internal server error instead of a clean HTTP 400 validation error.

Root Cause: The prescription validation model allowed arbitrary strings without checking against allowed standard medical frequency codes (OD - Once Daily, BID - Twice Daily, TID - Thrice Daily, QID - Four times Daily).

Fix Applied: Updated Pydantic `MedicationItem` model to validate frequency against allowed set {"OD", "BID", "TID", "QID"} and throw an explicit HTTP 400 `HTTPException` with descriptive message if validation fails.

2. Patient Prescription Privacy Leak (Unauthorized Data Access)

Failing Test: `test_prescriptions.py::test_prescription_access_control`

Symptom: A logged-in patient could view another patient's medical prescription by supplying the prescription UUID in GET `/api/prescriptions/{id}`, returning HTTP 200 instead of HTTP 403.

Root Cause: The endpoint verified that the bearer JWT token was valid, but failed to assert that `prescription["patient_id"] == current_user["id"]` or that the requester had an administrative/doctor role.

Fix Applied: Added explicit authorization checks to enforce that only the patient who owns the prescription, the prescribing doctor, or an admin can access the prescription resource. Otherwise, an HTTP 403 Forbidden exception is returned.

Post Testing Improvements

1. OPD Delay & Running Late Notifications:

Testing queue scenarios highlighted that when consultations take longer than expected, patients are left waiting without visibility. Added a new endpoint POST `/api/doctor/set-late` so doctors can mark themselves running late with an estimated delay (e.g., 40 mins), automatically updating live queue cards for all waiting patients.

2. Automated Wallet Refund Ledger:

Integration testing confirmed that cancellations require strict audit trails. Updated booking cancellation logic to log explicit refund entries in patient wallet history with timestamps and transaction reference IDs.

3. Structured Medication Schema for Print/PDF:

Standardized medication object structures to include explicit fields for `food_instructions` ("before" / "after") and `duration_days`, enabling clean frontend PDF generation and printable prescription layouts.

User Feedback and Plan for the Next Sprint

After demonstrating the implemented NirogPath application to key target users (Patients, Doctors, and Hospital Reception staff), the following feedback was recorded:

Feedback on the Patient OPD & E-Prescription Experience:

Patients loved seeing their live token queue number and estimated waiting time. However, patients noted that after their consultation, they often forget specific dietary advice or medication timing instructions. They suggested having an interactive assistant on their prescription screen to ask questions like *"Should I take Paracetamol after food?"* or *"What foods should I avoid with this medicine?"*.

Feedback on Doctor OPD Queue Management:

Doctors appreciated the single-click **"Call Next Patient"** workflow. They noted that during busy clinic hours, patients frequently ask receptionist staff about queue delays. Having an automated WhatsApp/SMS alert triggered when a patient's token is 2 positions away would reduce waiting room crowding and receptionist queries.

Feedback on Reception & Hospital Kiosk:

Hospital administrators highlighted that elderly patients or walk-in patients without smartphones need a dedicated Receptionist Kiosk interface to quickly generate paper tokens and accept cash deposits without requiring an app login.

Improvement Plan for Sprint 2 (Milestone 4):

- **AI-Powered Prescription Q&A Assistant (RAG Chatbot):** Integrate an AI assistant into the prescription detail view to answer patient questions regarding prescribed medications, dosage timings, and precautions.
- **Automated Queue SMS / WhatsApp Alerts:** Add Twilio integration to send automated SMS/WhatsApp notifications when a patient's turn is approaching (token position ≤ 2).
- **Receptionist Offline Walk-in Kiosk:** Implement a simplified reception interface for rapid walk-in booking and cash wallet top-ups.